Cambridge (CIE) A Level Computer Science

Your notes

File Organisation & Access

Contents

- File Organisation
- File Access
- Hashing

© 2025 Save My Exams, Ltd.

File Organisation

Your notes

- Modern computer systems **handle vast amounts of data**
- Everything from **bank transactions** to **music libraries** is stored in **files**
- To retrieve information quickly and efficiently, data must be **organised properly**

What is file organisation?

- **Files** are used to **store data permanently** on secondary storage
- Data in a file is stored as a **collection of records**, where each record represents a single item (e.g. a customer, a transaction, a product)
- The way records are arranged within a file is called the file's **organisation method**
- Choosing the right method affects how quickly data can be **searched, added, updated, or deleted**

Why is file organisation important?

Reason	Explanation
Efficient data access	**Well-organised** files allow fast searching and retrieval
Supports large-scale systems	Enables millions of users to **access data simultaneously** (e.g. online banking)
Improves program performance	**Reduces load times** and processing requirements
Enables reliable updates	Helps **manage changes** to stored data safely and accurately

File organisation methods

What are the different file organisation methods?

- The way data is arranged within a file **affects how efficiently it can be accessed** and **updated**
- Each file organisation method has **its own strengths** and is **suited to different types of tasks**

Serial file organisation

Description	Example use case

© 2025 Save My Exams, Ltd.

Records are stored **in the order they arrive** (no sorting). To find data, the file must be searched from the beginning.	Logging sensor data, storing queued messages

- **Simple to implement**, but **slow to search** if there are many records

Sequential file organisation

Description	Example use case
Records are stored **in a sorted order** based on a key field. Searching can stop once the target is passed.	Payroll systems, exam results

- **More efficient for searches** and ideal for files that are **read often but rarely updated**

Direct (random) file organisation

Description	Example use case
Records are placed at specific locations using a **hashing algorithm**. This allows **instant access** to any record.	Bank account access, real-time stock updates

- Best for **fast, real-time access** to individual records

Your notes

Your notes

File access methods

What are file access methods?

- Once a file is organised and stored, it must be accessed correctly **in order to retrieve or update data**
- The method of access depends on the **file organisation** used

Sequential access

- Sequential access means accessing records **one after another, in order**, starting from the beginning of the file

Key features	Used with
Reads records in **sequence**	**Serial** and **Sequential** files
Slower for searching **large files**	Must process **earlier records first**
Efficient when reading the **whole file**	Ideal for **batch processing** (e.g. payroll, reports)

- **Example**: Reading every student's exam result in order of candidate number

Direct (random) access

- Direct access allows a program to **jump straight to the location** of a specific record, without reading previous ones

Key features	Used with
Accesses records **directly by position or key**	**Sequential** (with indexing) and **Random** files
Fast access to individual records	Ideal for **databases and real-time systems**
Requires a **calculation or index** to find the correct location (e.g. hashing)	Useful when you need to **look up a record quickly**

- **Example**: Accessing a bank account record by account number

© 2025 Save My Exams, Ltd.

Your notes

Hashing algorithms

What is a hashing algorithm?

- A hashing algorithm is used to calculate the **storage location (address)** of a record in a **random (direct access) file**
- It allows for **fast access** to data without searching sequentially through a file

Why use hashing?

- Provides **direct access** to records using a key (e.g. ID, account number)
- Useful when dealing with **large files** that need fast search, read, or write operations
- Used in **random files** and sometimes in **indexed sequential files**

How hashing works?

- A hashing algorithm takes a **key** (e.g. StudentID) and uses a **mathematical function** to calculate a **file address**

```
Address = HashFunction(Key)
```

- If two keys produce the same address, a **collision** occurs
- This must be resolved (e.g. via **linear probing** or **overflow areas**)

Writing a record to a file:

```
Address ← HashFunction(Key)
WRITE file[Address], Record
```

Reading a record from a file:

```
Address ← HashFunction(Key)
READ file[Address], Record
```

- If the expected record is **not found**, collision handling must be applied to find the correct record

Types of hashing algorithms

Method	Description
Modulo Division	Most common method: Key MOD TableSize
Folding Method	Breaks the key into parts and adds them together
Mid-Square Method	Square the key and use the middle digits as the address

© 2025 Save My Exams, Ltd.

Multiplicative Hashing	Multiplies the key by a constant and uses fractional part × tablesize

Your notes

Collision handling strategies

Strategy	Description
Linear probing	Try the next available slot in sequence
Overflow area	Store conflicting records in a separate file/area
Chaining	Store all records with the same address in a list

© 2025 Save My Exams, Ltd.